Data 37711 Homework 2 — Problem 2

Encoding high-cardinality categorical features

Yiwen Chen

The code is in `problem2.py`. All numbers below come from one run with seed 0.

Setup: target transformation and loss

`Quantity` is a positive integer with a very heavy right tail:

$$\text{median} = 6, \quad q_{0.9} = 24, \quad q_{0.99} = 120, \quad q_{0.999} = 504, \quad \max = 80{,}995,$$

with mean 13.0 and standard deviation 180. With squared error on the raw scale, a handful of huge orders would dominate the loss and the gradients. Since $\texttt{Quantity} \geq 1$, I instead model

$$t = \log(\texttt{Quantity}) \in [0,\ 11.3],$$

and train every model with mean squared error on t:

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\big(t_i - \hat{t}_\theta(x_i)\big)^2.$$

This is the maximum-likelihood loss for the model $\log Y \mid x \sim \mathcal{N}(\hat{t}(x), \sigma^2)$, i.e. a log-normal model for `Quantity`. Large orders count in relative rather than absolute terms: predicting 60 for a true value of 120 costs the same as predicting 3 for a true value of 6. The prediction $\exp(\hat{t}(x))$ estimates the conditional *median* (geometric mean) of `Quantity`, which is a sensible point forecast for a skewed target.

Metrics. On the test set I report RMSE, MAE and R^2 on the log scale (the training objective), and the MAE and median absolute error of $\exp(\hat{t})$ in original units.

(a) Cardinality and data split

Feature	# unique	rows/level (median)	min	max	levels with < 5 rows
`StockCode`	3,665	49	1	2,035	472
`CustomerID`	4,339	41	1	7,847	228

Table 1: Cardinality of the two features (n = 397,924 rows; 266,802 distinct (item, customer) pairs).

Both features have *high cardinality*: thousands of levels, many of them rare. This has three consequences.

- **One-hot encoding is feasible, but only as a sparse matrix.** It produces 3,665+4,339 = 8,004 columns. A dense `float32` design matrix would take 397,924×8,004×4 bytes ≈ 12.7 GB. A sparse matrix has only 2 non-zeros per row ($\approx 8\times10^5$ in total), so a linear model on one-hot features is cheap to fit. I fit it as a baseline below. However, one-hot features give each level its own parameter that is learned in isolation: there is no sharing of information between similar items or customers, rare levels get noisy estimates, and a linear model on one-hot features can only express additive effects $a_{\text{item}} + b_{\text{customer}}$. Adding all pairwise interactions would require $\approx 1.6 \times 10^7$ columns, which is infeasible.
- **Compact encodings are attractive.** *Target encoding* maps each level to a single number. *Learned embeddings* map each level to a dense d-dimensional vector with $d \ll$ cardinality.
- **Rare and unseen levels need care.** Shrink target encodings towards the global mean, and map rare or unseen levels to a shared "unknown" embedding.

Split. I split the rows at random into 70% train (278,546), 15% validation (59,688) and 15% test (59,690). The validation set is used for early stopping and for choosing the embedding dimension. All encodings (target-encoding statistics and embedding vocabularies) are computed from the *training set only*. Only 0.09% (`StockCode`) and 0.04% (`CustomerID`) of test rows have a level that never appears in training. 42% of test rows have an (item, customer) pair that also appears in training.

(b) Target encoding

Encoding. For a level c of a feature, let n_c be its number of training rows and $\bar{t}_c$ its mean of t, and let $\bar{t}$ be the global training mean. I use the smoothed target encoding

$$\text{TE}(c) = \frac{n_c\,\bar{t}_c + m\,\bar{t}}{n_c + m} = \lambda_c\,\bar{t}_c + (1-\lambda_c)\,\bar{t}, \qquad \lambda_c = \frac{n_c}{n_c+m}, \quad m = 20. \tag{1}$$

This is the posterior mean of the level's mean under a Gaussian prior centred at $\bar{t}$. The prior has the weight of m pseudo-observations, so a level seen once is pulled almost all the way to $\bar{t}$, while a level seen 2,000 times keeps its own mean. Unseen levels get $\bar{t}$. I encode the mean of $t = \log$ `Quantity`, not of raw `Quantity`, because a raw mean would be dominated by single huge orders.

Avoiding target leakage. If a training row's own target is used to compute its feature, then $\text{TE}(c)$ contains $t_i/(n_c+m)$, which is a direct leak for rare levels, and the model learns to trust TE too much. I therefore compute the encoding of *training* rows out-of-fold. The training set is split into 5 folds, and rows in fold k are encoded with statistics from the other 4 folds. Validation and test rows are encoded with statistics from the full training set. (`category_encoders.TargetEncoder` uses the same smoothing idea; I wrote it directly in `pandas` to control the out-of-fold step.)

Models. With the two encoded features ($\text{TE}_{\text{item}}, \text{TE}_{\text{cust}}$) as input (standardized), I fit (i) a linear regression and (ii) an MLP with hidden layers (128, 64), ReLU, dropout 0.1, AdamW (lr 2×10^{-3}, batch 1024), and early stopping on validation MSE (patience 4). The out-of-fold TE features have correlation 0.48 (item) and 0.68 (customer) with t.

Results are in Table 2. TE→MLP reaches a test RMSE of 0.712 on the log scale ($R^2 = 0.652$). This is much better than the global mean ($R^2 = 0$) but, interestingly, *worse* than the one-hot additive baseline (0.673); see (e).

(c) Embedding model

Architecture. Each feature gets its own embedding table, $E_{\text{item}} \in \mathbb{R}^{K_{\text{item}} \times d}$ and $E_{\text{cust}} \in \mathbb{R}^{K_{\text{cust}} \times d}$. Row 0 of each table is shared by all levels that are unseen or rare (fewer than 3 training rows), which gives $K_{\text{item}} = 3{,}258$ and $K_{\text{cust}} = 4{,}145$. For one transaction,

$$\hat{t}(x) = \text{MLP}\big([\, E_{\text{item}}[s(x)] \;;\; E_{\text{cust}}[c(x)] \,]\big) \in \mathbb{R},$$

where the input to the MLP is the concatenation of the two embeddings ($2d$ dimensions). The MLP head and the training recipe are identical to (b), so the only difference between the models is the input representation. The embeddings are trained end-to-end with the MLP.

Choice of d. I swept $d \in \{4, 8, 16, 32\}$ and chose the value with the lowest validation MSE:

d	4	8	16	32
best validation MSE (log)	0.3991	0.3969	**0.3933**	0.3952

The curve is flat: any d between 4 and 32 works. I use $d = 16$, which is about $\sqrt[3]{K}$-$\sqrt{K}$ for these vocabularies and has $16 \times (3{,}258 + 4{,}145) \approx 1.2 \times 10^5$ embedding parameters, comfortably fewer than the 278k training rows.

Result. The embedding model reaches a test RMSE of 0.628 ($R^2 = 0.729$). This is a clear improvement over target encoding (0.712, i.e. 22% lower MSE) and over the one-hot additive model (0.673). In original units, the median absolute error falls from 1.82 items (TE) to 1.29 items.

(d) Target encoding + embeddings

The input is now $[\, E_{\text{item}}[s] \;;\; E_{\text{cust}}[c] \;;\; \text{TE}_{\text{item}} \;;\; \text{TE}_{\text{cust}} \,] \in \mathbb{R}^{2d+2}$, with the same MLP head and $d = 16$. Because the TE features of training rows are out-of-fold, the network cannot use them as a leak of t_i.

Model	RMSE (log)	MAE (log)	R^2 (log)	MAE (Qty)	MedAE (Qty)
Global mean (baseline)	1.2077	1.0013	0.000	10.37	4.20
One-hot additive, linear (baseline)	0.6733	0.4990	0.689	6.54	1.61
(b) TE → linear	0.7284	0.5522	0.636	7.10	1.93
(b) TE → MLP	0.7122	0.5331	0.652	6.90	1.82
(c) Embeddings ($d = 16$) → MLP	0.6283	0.4408	0.729	5.92	1.29
(d) TE + Embeddings → MLP	**0.6212**	**0.4359**	**0.735**	**5.81**	**1.23**

Table 2: Test-set results ($n = 59{,}690$). The target is $t = \log \texttt{Quantity}$; the "Qty" columns compare $\exp(\hat{t})$ with the raw quantity.

The combined model is the best on every metric, but its gain over embeddings alone is small: test RMSE 0.628 → 0.621 (about 2% lower MSE), and validation MSE 0.3933 → 0.3875. The TE features add little information beyond what the embeddings can learn. Their main benefits are that the network does not need to discover the main effects itself (it converged in 9 epochs instead

of 18; Figure 1), and that they help most for *rare customers*: in the 1–10-row bucket of Table 4, RMSE drops from 0.704 to 0.679. For those customers the shrunk TE value is a better summary than an embedding trained on a few examples.

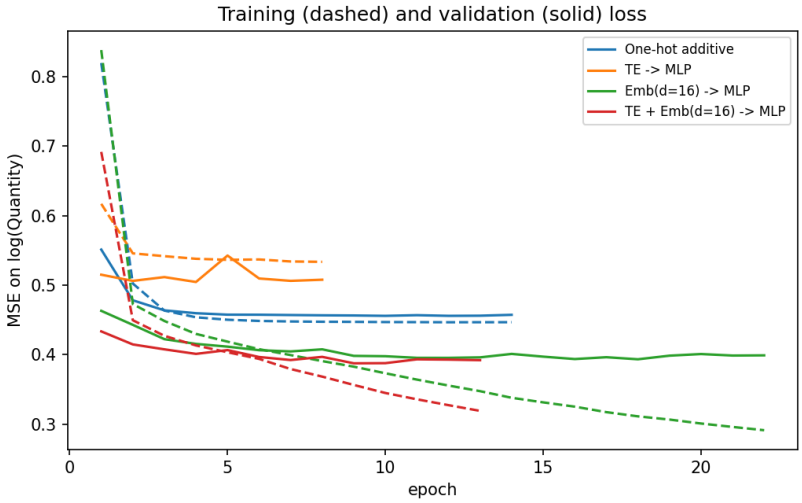


Figure 1: Training (dashed) and validation (solid) MSE for the neural models.

(e) Intuition and supporting evidence

What the embedding layer is. Let $e_c \in \{0,1\}^K$ be the one-hot vector of level c. An embedding lookup is a linear layer applied to the one-hot vector:

$$E[c] = E^\top e_c, \qquad E \in \mathbb{R}^{K \times d}.$$

An embedding layer is therefore *exactly* one-hot encoding followed by a trainable linear map to $\mathbb{R}^d$, implemented as a lookup so the sparse matrix is never formed. Two special cases put the other methods in the same framework:

- With $d = 1$ and the two scalars *summed* (no MLP), we get $\hat{t} = b + a_{\text{item}} + b_{\text{cust}}$, which is precisely linear regression on one-hot features. This is how I fit the one-hot baseline.
- Target encoding is a $d = 1$ "embedding" that is *fixed in advance* to each level's marginal mean of t, rather than learned for the task.

A learned $d = 16$ embedding is a strict generalization of both. Each item and each customer gets a 16-dimensional vector of latent "traits", learned jointly with the MLP to minimize the loss.

Why it helps here.

1. **Joint rather than marginal effects.** TE uses each feature's *marginal* mean. These means are confounded: a wholesale customer buys many bulk items, so the items they buy have inflated marginal means, and vice versa. The model then double-counts. The one-hot additive model estimates both effects *jointly*, which is why it already beats TE→MLP (0.673 vs. 0.712) even though it is linear. Embeddings are also learned jointly.

2. **One number per level is too little.** After target encoding, two items with the same mean quantity are indistinguishable, even if one is bought in bulk by wholesalers and the other in ones and twos by everybody. An MLP on $(\mathrm{TE}_{\mathrm{item}}, \mathrm{TE}_{\mathrm{cust}})$ can only learn a function of two scalars, and it gains little over the linear model on TE (0.712 vs. 0.728).

3. **Interactions.** With d-dimensional vectors, the MLP can learn *customer×item interactions*, e.g. "this customer buys *this kind* of item in bulk". That is a low-rank factorization of the $4{,}339 \times 3{,}665$ interaction table, in the spirit of matrix factorization for recommender systems. One-hot features would need 1.6×10^7 interaction columns for this, and TE cannot express it at all.

Evidence on the test set. *(i) Models use interactions only when given embeddings.* For each model I fitted the best additive approximation $a_{\mathrm{item}} + b_{\mathrm{cust}}$ (by backfitting) to that model's own test predictions. The fraction of prediction variance that is *not* additive is:

	One-hot additive	TE→MLP	Emb→MLP	TE+Emb→MLP
non-additive share of $\mathrm{Var}(\hat{t})$	0.000	0.018	0.087	0.074

The TE model is essentially additive. The embedding models put about 8–9% of their prediction variance into interaction effects, and they are also the models with the lowest test error.

(ii) Performance by how often a level is seen. Tables 3 and 4 break down the test RMSE (log) by the number of training rows of the test row's item or customer.

train rows of item	n_{test}	Global	One-hot add.	TE→MLP	Emb→MLP	TE+Emb
0 (unseen)	53	1.277	0.720	0.820	**0.701**	0.736
1–10	929	1.276	0.749	0.847	0.747	**0.745**
11–50	6,694	1.261	0.693	0.753	0.679	**0.664**
51–200	25,416	1.196	0.669	0.701	0.624	**0.613**
> 200	26,598	1.203	0.669	0.707	0.614	**0.612**

Table 3: Test RMSE (log) by frequency of the `StockCode` in training.

train rows of customer	n_{test}	Global	One-hot add.	TE→MLP	Emb→MLP	TE+Emb
0 (unseen)	21	2.206	**1.142**	1.833	1.282	1.149
1–10	1,298	1.338	0.711	0.935	0.704	**0.679**
11–50	10,802	1.169	0.625	0.695	0.588	**0.571**
51–200	24,519	1.150	0.632	0.666	0.584	**0.577**
> 200	23,050	1.275	0.732	0.750	0.684	**0.682**

Table 4: Test RMSE (log) by frequency of the `CustomerID` in training.

The embedding models win in every bucket with enough data. Their advantage over TE is largest for frequent levels (e.g. customers with 11–50 rows: 0.588 vs. 0.695), where there is enough data to learn a 16-dimensional profile. For very rare levels, all methods fall back to something close to a main effect, and the shrunk TE value is a useful extra input (the TE+Emb column). The "unseen" buckets contain only 53 and 21 rows, so their differences are noisy.

(iii) Pairs seen and not seen together. 42% of test rows are an (item, customer) pair that also occurs in training. The embedding models improve on TE→MLP for both seen pairs (0.752 → 0.662) and *unseen* pairs (0.682 → 0.586). The gain on unseen pairs shows that the improvement is not memorization of specific pairs. The model generalizes to new combinations through the shared latent space, which is exactly what a factorization model should do.

(iv) Geometry of the learned embeddings. Figure 2 shows the first two principal components of the learned embeddings, coloured by each level's training mean of t. The first principal component is strongly correlated with mean log quantity: $|\rho| = 0.86$ for items and 0.91 for customers. So the main direction the network learns is essentially a target encoding ("buys/is bought in large quantities"), learned from the loss rather than computed in advance. The remaining directions (about half of the embedding variance) carry information that is *not* the mean, and that information is what the interactions use. Nearest neighbours also look meaningful. For example, item `85099B` is closest to `23199`, `23202`, `23203` and `85099F`, a family of neighbouring codes (very likely variants of the same product line) with similar typical order sizes. The embedding therefore groups related products, even though the model was never told that their codes are related.

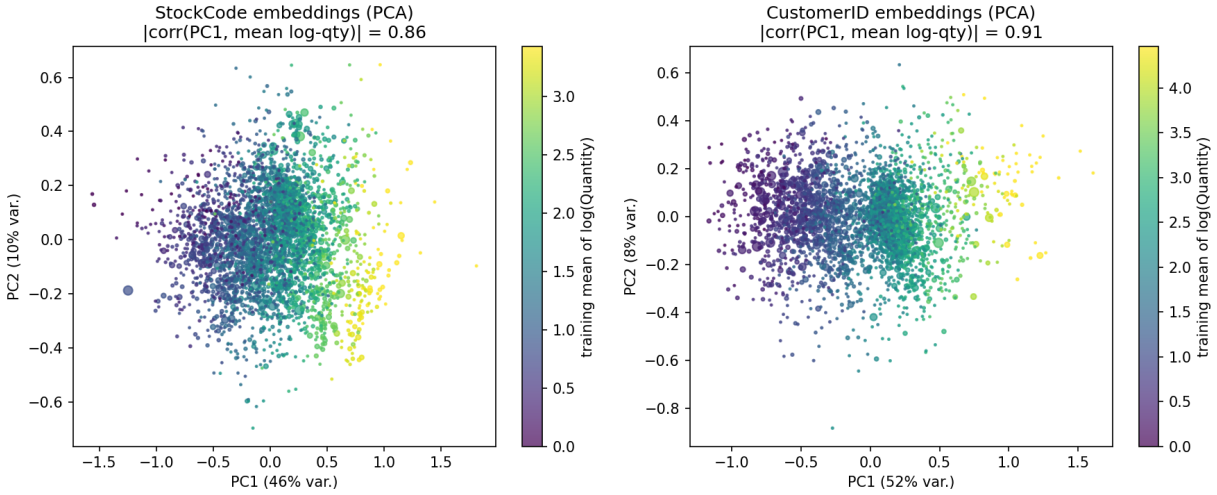


Figure 2: PCA of the learned $d = 16$ embeddings (rows with ≥ 3 training transactions), coloured by the training mean of $\log$ `Quantity`; marker size $\propto$ frequency.

Summary. Target encoding compresses each level into one fixed marginal statistic. It is cheap and robust for rare levels, but it is confounded across features and cannot represent interactions. Learned embeddings are one-hot encoding followed by a learned low-dimensional projection. They estimate effects jointly, share statistical strength between similar levels, and let the network learn low-rank item×customer interactions, which gives the largest gain (test MSE on the log scale: TE 0.507, embeddings 0.395, combined 0.386). Combining the two helps slightly, mainly for rare customers, because the shrunk TE value is a good summary where the embedding has seen too little data.